University of Sri Jayewardenepura

ICT 1051 Database Systems & Administration

First Year – Second Semester Individual Final Assignment - 2025

Name: R. K. D. A. Kasundi

Index number: AS20240980

Acknowledgment

I would like to express my sincere gratitude to Dr. Ilmini for her guidance on database design module, which helped to understand relational modeling and ER diagram construction more clearly.

My family deserves my deepest appreciation because they maintained their support for me which enabled me to stay dedicated to my work.

Contents

1. Introduction

The report presents complete details about the database component of the Sales Management System which was created for ICT 1062. The system functions as a desktop application developed with Java programming language and NetBeans IDE while MySQL serves as its Relational Database Management System (RDBMS).The sales_db database contains all application data which needs to remain accessible throughout the application lifecycle including user credentials and customer records and product catalogue information and sales transaction records.

The database design implements relational database principles through foreign key constraints which maintain referential integrity and CHECK constraints which guarantee data accuracy. The application layer implements the Data Access Object (DAO) design pattern to achieve database logic separation from user interface components.

2. Database Overview

2.1 Settings and Technology

The following technologies and settings are used for the database:

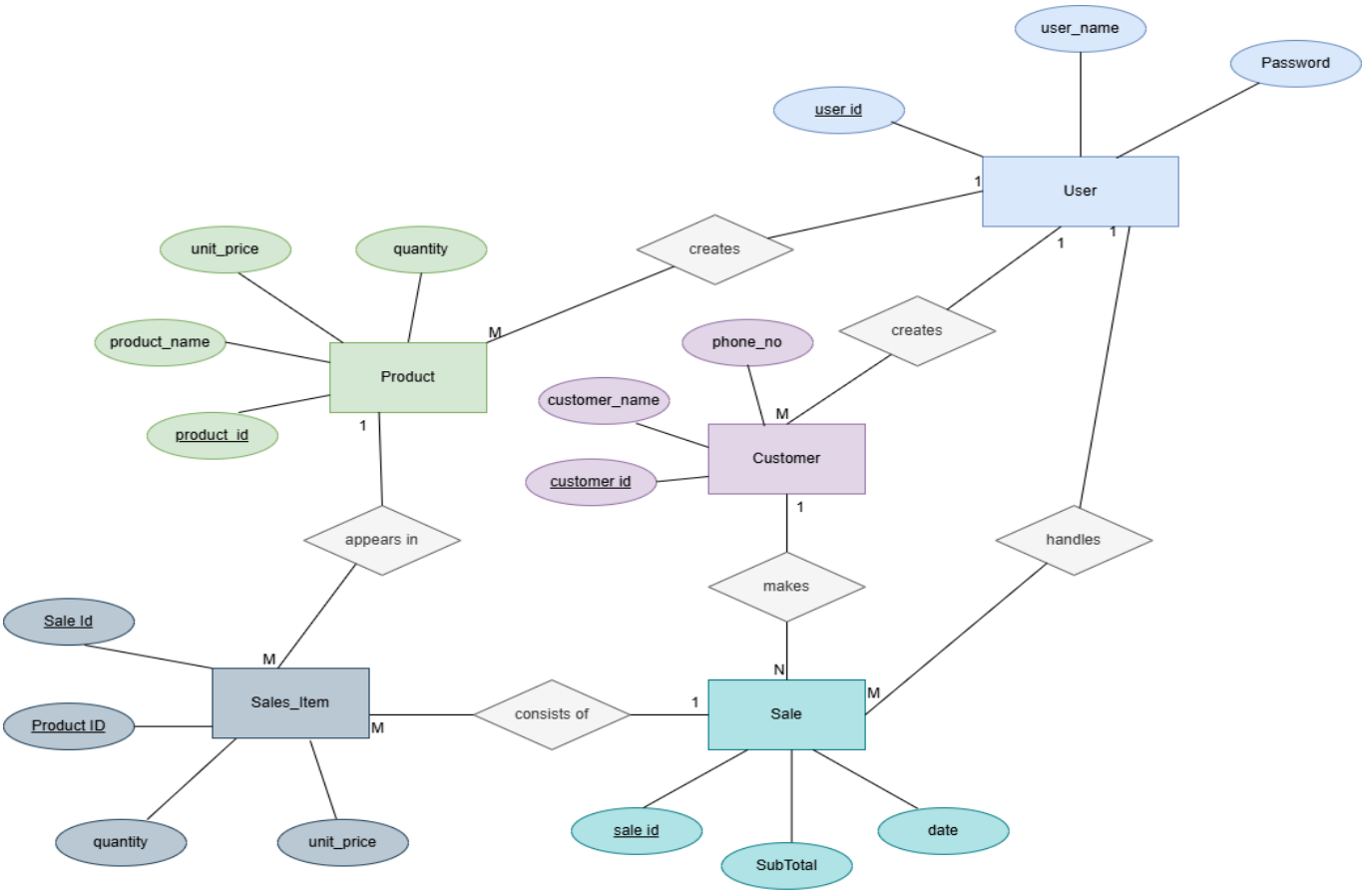
Components	Details
RDBMS	MySQL 8.x
Database Name	sales_db
Host	localhost
Port	3308
JDBC Driver	MySQL Connector/J (jdbc:mysql://)
Application Language	Java (NetBeans IDE)
DAO Pattern	CustomerDAO, ProductDAO, saleDAO, LoginController

2.2 Database Initialization

The database is created and initialized using the SQL script sales_db.sql. The script performs the following steps in order:

- Creates the database: CREATE DATABASE sales_db
- Selects it for use: USE sales_db
- Creates each table with all columns, data types, and constraints
- Inserts sample seed data (users, customers, and products)

3. Entity Relationship Diagram



The database models five entities with the following relationships:

- users → customer (one-to-many: a user creates/updates many customers)
- users → product (one-to-many: a user manages many products)
- users → sale (one-to-many: a user processes many sales)
- customer → sale (one-to-many: a customer has many sales)
- sale → sale_item (one-to-many: a sale contains many items)
- product → sale_item (one-to-many: a product appears in many sale items)

4. Table Definitions

The database contains five tables.

Table	Purpose	Relationships
users	System login credentials	Referenced by customer, product, sale
customer	Customer contact details	FK to users; referenced by sale
product	Product catalogue & stock levels	FK to users; referenced by sale_item
sale	Sales transaction header	FK to customer, users; referenced by sale_item
sale_item	Line items of each sale	FK to sale, product (composite PK)

4.1 Table : users

Stores system user accounts. Each user can log in to the application using a username and password combination. The table also acts as a foreign key reference for audit columns in the customer and product tables.

Column	Data Type	Constraints	Description
user_id	`INT`	PK, AUTO_INCREMENT	Unique identifier for each system user
user_name	`VARCHAR(25)`	UNIQUE, NOT NULL	The login username; must be unique across all users
password	`VARCHAR(255)`	NOT NULL	The user's password; stored in plain text (noted security concern – see Section 5)

Query Used for Authentication

```
SELECT * FROM users WHERE user_name = ? AND password = BINARY ?;
```

The BINARY keyword enforces case-sensitive password comparison. PreparedStatement is used to parameterise the inputs and prevent SQL injection.

4.2 Table: customer

Stores information about customers who make purchases. A customer is uniquely identified by their phone number. The created_by column records which user added the customer to the system, and created_date is automatically set to the current timestamp on insertion.

Column Name	Data Type	Constraints	Description
customer_id	`INT`	PK, AUTO_INCREMENT	Auto-generated unique identifier for each customer
customer_name	`VARCHAR(255)`	NOT NULL	Full name of the customer
phone_no	`VARCHAR(20)`	NOT NULL, UNIQUE	Customer's phone number; used as the natural unique identifier
created_by	`INT`	FK → users(user_id)	The system user who created this customer record
created_date	`DATETIME`	NOT NULL, DEFAULT NOW()	Timestamp automatically set when the record is created

DAO Operations (Customer DAO)

- addCustomer(cName, phone, userId) – inserts a new customer and returns the generated customer_id
  - `INSERT INTO customer (customer_name, phone_no,created_by) VALUES (?,?,?);`
  - `SELECT customer_id FROM customer WHERE phone_no = ?;`

- viewCustomerList() – retrieves all customers (id, name, phone)
  - `String sql = "SELECT customer_id, customer_name, phone_no FROM customer";`

- getCustomerByPhone(phone) – looks up a customer by their phone number for use during the sale process
  - `SELECT customer_id, customer_name, phone_no FROM customer WHERE phone_no = ?;`

4.3 Table: product

Stores the product catalogue along with real-time stock quantities. Soft deletion is implemented via the is_active flag; deleted products are never physically removed from the database so that historical sales records remain intact.

Column Name	Data Type	Constraints	Description
product_id	`INT`	PK, AUTO_INCREMENT	Auto-generated unique identifier for each product
product_name	`VARCHAR(255)`	NOT NULL	Descriptive name of the product
unit_price	`DECIMAL(7,2)`	NOT NULL, CHECK (unit_price > 0)	Selling price per unit; must be positive
quantity	`INT`	NOT NULL, CHECK (quantity >= 0)	Current stock level; cannot be negative
updated_by	`INT`	FK → users(user_id)	The system user who last modified this product record
created_date	`DATETIME`	DEFAULT NOW()	Timestamp set at the time of product creation
updated_date	`DATETIME`	DEFAULT NOW() ON UPDATE NOW()	Automatically updated to the current timestamp on every row modification
is_active	`TINYINT(1)`	NOT NULL, DEFAULT 1	Soft-delete flag: 1 = active, 0 = deleted. Deleted products are excluded from all UI listings but retained for historical data.

DAO Operations (ProductDAO.java)

- addProduct(pName, unitPrice, quantity, userId) – inserts a new product
  - `INSERT INTO Product (product_name,unit_price,quantity,updated_by) VALUES (?,?,?,?);`
- setProduct(productId, userId) – retrieves a single product by ID for the update form
  - `SELECT product_name, unit_price, quantity FROM product WHERE product_id = ?;`
- updateProduct(pName, unitPrice, quantity, userId, productId) – updates all editable fields

- `Update product SET product_name = ?, unit_price = ?, quantity = ? , updated_by = ? WHERE product_id = ?;`

- deleteProduct(productId) – performs a soft delete by setting is_active = 0
  - `UPDATE product SET is_active = 0 WHERE product_id = ?;`

- viewProductList() – retrieves only active products (is_active = 1) for display and sale
  - `SELECT product_id, product_name, unit_price, quantity FROM product WHERE is_active = 1;`
- updateStock(productId, quantitySold) – decrements the quantity by the amount sold
  - `UPDATE product SET quantity = quantity - ? WHERE product_id = ?;`

4.4 Table: sale

Stores the header record for each sales transaction. Each row represents one complete sale made by a specific user to a specific customer. The sub_total records the total monetary value of the transaction.

Column Name	Data Type	Constraints	Description
sale_id	`INT`	PK, AUTO_INCREMENT	Auto-generated unique identifier for each sale transaction
customer_id	`INT`	NOT NULL, FK → customer	The customer who made the purchase
user_id	`INT`	NOT NULL, FK → users	The system user (cashier) who processed the sale
sub_total	`DECIMAL(9,2)`	NOT NULL	Total monetary value of all items in the sale
sale_date	`DATETIME`	NOT NULL, DEFAULT NOW()	Date and time at which the sale was recorded

DAO Operations (saleDAO.java)

- saveSale(int customerId, int userId, double subTotal, List<SaleItem> items) – insert one row into the sale table
  - ```
    INSERT INTO sale (customer_id, user_id, sub_total, sale_date)
    VALUES (?, ?, ?, NOW());
    ```
  - ```
    INSERT INTO sale_item (sale_id, product_id, quantity, unit_price)
    VALUES (?, ?, ?, ?);
    ```

- getSales() – pulls data from tables(sale, customer, user)
  - ```
    SELECT s.sale_id, s.sub_total, s.sale_date,
                        c.customer_id, c.customer_name, c.phone_no,
                        u.user_id, u.user_name
                        FROM sale s
                       JOIN customer c ON s.customer_id = c.customer_id
                       JOIN users u ON s.user_id = u.user_id
                       ORDER BY s.sale_date DESC;
    ```

- getSaleItems(int saleId) – get all items belonging to one specific sale
  - ```
    SELECT p.product_id, p.product_name, si.quantity, si.unit_price
                        FROM sale_item si
                        JOIN products p ON si.product_id = p.product_id
                        WHERE si.sale_id = ?;
    ```

4.5 Table: sale_item

Stores the individual line items that belong to each sale. A composite primary key (sale_id, product_id) ensures that each product can appear at most once per sale. The unit_price is stored at the time of the sale to preserve pricing history even if the product's current price is later changed.

Column Name	Data Type	Constraints	Description
sale_id	`INT`	PK (composite), FK → sale	References the parent sale transaction
product_id	`INT`	PK (composite), FK → product	References the product being sold
quantity	`INT`	NOT NULL, CHECK (quantity > 0)	Number of units of this product sold in this transaction
unit_price	`DECIMAL(7,2)`	NOT NULL	Price per unit at the time of the sale (snapshot for historical accuracy)

5. Constraints & Data Integrity

5.1 Primary Keys

Every table has a clearly defined primary key. Four tables use a single AUTO_INCREMENT integer column as a surrogate primary key. The sale_item table uses a composite primary key (sale_id, product_id), which naturally prevents the same product from being added twice to the same sale.

5.2 Foreign Keys

Foreign key constraints enforce referential integrity across the following relationships:

Table	Column	References	Meaning
customer	created_by	users(user_id)	Records who registered the customer
product	updated_by	users(user_id)	Records who last modified the product
sale	customer_id	customer(customer_id)	Identifies the purchasing customer
sale	user_id	users(user_id)	Identifies the cashier who made the sale
sale_item	sale_id	sale(sale_id)	Links the line item to its parent sale
sale_item	product_id	product(product_id)	Links the line item to the product sold

5.3 CHECK Constraints

CHECK constraints protect against invalid data entry at the database level, independent of application-layer validation:

- product.unit_price > 0 – ensures a product cannot have a zero or negative price
- product.quantity >= 0 – ensures stock levels cannot go below zero
- sale_item.quantity > 0 – ensures at least one unit must be present on a line item

5.3 `UNIQUE` Constraint

- users.user_name – prevents duplicate login names
- customer.phone_no – ensures each phone number is registered to only one customer

5.4 `DEFAULT` values & `TIMESTAMPS`

- customer.created_date DEFAULT NOW() – set automatically on insertion
- product.created_date DEFAULT NOW() – set automatically on insertion
- product.updated_date DEFAULT NOW() ON UPDATE NOW() – automatically updated on every row change
- sale.sale_date DEFAULT NOW() – set automatically to the current date/time at point of sale

6. Conclusion

The database design of the Sales Management System is well-structured and follows sound relational database principles. The five-table schema cleanly separates concerns across users, customers, products, sales, and sale line items. Referential integrity is maintained through foreign keys, and data correctness is enforced by CHECK and UNIQUE constraints. The use of the DAO pattern, PreparedStatement, and JDBC transactions in the application layer provides a solid and secure data access architecture suitable for the scope of an introductory programming project.

Key areas identified for improvement in a production scenario include password hashing, externalisation of database credentials, and connection pooling. These improvements would bring the implementation in line with industry standards for security and scalability.